0048. Rotate Image

edium | Category: Math & Geometry | LeetCode: https://leetcode.com/problems/r

Problem

- Rotate an n x n matrix 90 degrees clockwise, in place.

Intuition

- A clockwise rotation equals a transpose (swap across the main diagonal) followed by reversing each row.

Brute Force

- Copy into a new matrix with res[c][n-1-r] = m[r][c]: O(n^2) extra space, which is not allowed.

Optimal Approach

- Transpose: for r < c swap m[r][c] and m[c][r].
- Reverse every row.

Worked Example

```
1 2 3     transpose     1 4 7     reverse rows     7 4 1
4 5 6        ->         2 5 8          ->          8 5 2
7 8 9                   3 6 9                      9 6 3
```

Java Solution

```java
public class RotateImage {
    public void rotate(int[][] m) {
        int n = m.length;
        for (int r = 0; r < n; r++)
            for (int c = r + 1; c < n; c++) { int t = m[r][c]; m[r][c] = m[c][r]; m[c][r] = t; }
        for (int[] row : m)
            for (int l = 0, r = n - 1; l < r; l++, r--) { int t = row[l]; row[l] = row[r]; row[r] = t; }
    }
}
```

Complexity

- Time: O(n^2). Space: O(1).

Pitfalls

- In the transpose, only swap for c > r, or the second swap undoes the first.